Emmi Korhonen, Farnaz Jamilpanah

PawTracker – Mobile Application

Project Plan

Metropolia University of Applied Sciences

Mobile Development Project

Course code + implementation number

TX00EY39-3006

25.3.2026

Contents

1 Project Overview 2

2 Target Users 3

3 Project Goals 3

4 Success Metrics 4

5 Core Features 4

5.1 Optional Features 4

6 Technology Stack and Tools 5

7 Application Architecture 6

7.1 Folder Structure 7

8 Design 7

8.1 Application Layout 8

8.2 Logo 8

9 Scrum Process 8

10 Team Roles 9

11 Testing Plan 10

12 Risks and Mitigation 10

13 Timeline 13

1 Project Overview

The PawTracker – Dog Activity Tracker is an Android application designed to help dog owners monitor and improve their dog's daily physical activity. The main goal of the Dog Activity Tracker is to help dog owners understand and improve their dog's daily exercise habits. Many owners are unsure how much activity their dog gets. *This might affect negatively to dog's health.* This mobile application provides clear, accurate data to support healthier routines. Using the phone's internal GPS sensor, the app records walking routes, calculates distance and duration, and stores all activity data for long-term tracking. The purpose is to give owners a simple, visual, and motivating way to ensure their dogs receive enough exercise for a healthy lifestyle.

2 Target Users

The application is designed for pet owners who want to track their pet's daily exercise. Users are active dog owners who are careful or concerned about their pet's health and comfortable using smartphones. Application also caters to users who enjoy visual statistics and route history, as well as anyone looking for a simple, motivating activity tracker for their pets.

3 Project Goals

- Build a functional Android app within 6 weeks using the Scrum methodology.
- Learn and implement modern Android development practices, including MVVM architecture
- Create a visual and useful real-world application
- Use internal sensors(GPS) to collect real-time data
- Implement Room database
- Work with Google Maps API to visualize routes
- Integrate API to fetch data from web service

4 Success Metrics

- All core features (MVP) work as expected
- UI is accessible and follows Material Design
- Room database stores and retrieves data correctly
- Unit tests pass and application is tested with at least 50% coverage

5 Core Features

- **GPS tracking:** Records the pet's walking route in real time.
- **Map view:** Shows the route visually on a map.
- **Activity statistics:** Distance, duration and weekly totals.
- **History:** Stores past walks using Room database.
- **Design UI:** Clean, accessible, and responsive interface.
- **Unit tests:** Logic tests for ViewModel and data handling.

5.1 Optional Features

- **Dog profile:** Stores dog information and profile picture
- **Dog breed API:** Information about different breeds and their recommended activity levels

- **Health log:** Allows owners to record observations about the dog's droppings for health monitoring
- **Weather API:** Shows weather
- **Notifications:** Reminds the user when the dog hasn't been active enough
- **AI:** Provides simple weekly activity analysis

6 Technology Stack and Tools

Technology Stack

- **Programming Language:** Kotlin
- **UI Framework:** Jetpack Compose
- **Architecture:** MVVM (ViewModel + StateFlow)
- **Database:** Room
- **Sensors:** GPS (Location Services)
- **APIs:** Google Maps, OpenWeather API (optional), Dog API:
- **Testing:** JUnit (unit tests), UI tests

Tools

- **IDE:** Android Studio
- **Version Control:** GitHub
- **Project Management**: Jira
- **Design:** Figma, Adobe Firefly
- **Communication:** Discord

7 Application Architecture

The application follows the MVVM (Mode-View-Model) pattern. The responsibilities of different components are separated to make the code maintainable, testable and scalable. It follows Material Design principles and integrates internal sensors (GPS) and external APIs (such as weather data) to provide a clean, intuitive user experience.

UI Layer

- Contains all user interface components built with Jetpack Compose
- Responsible for displaying data and handling user interactions.
- Each screen has its own ViewModel and UiState to make the code easier to maintain and ensure separation.
- UI observes state changes from the ViewModel and updates automatically.

ViewModel Layer

- Acts as mediator between UI and the data layer.
- Handles business logic and prepares data for the UI.

- Handles user actions.

Data Layer

- Manages the data to ViewModel
- Local database Room stores walk history and statistics
- APIs fetches external data such as weather information

Data flows from the Data Layer to the ViewModel which processes it and exposes it to the UI Layer. User actions from the UI are passed to ViewModel that updates the data correctly.

7.1 Folder Structure

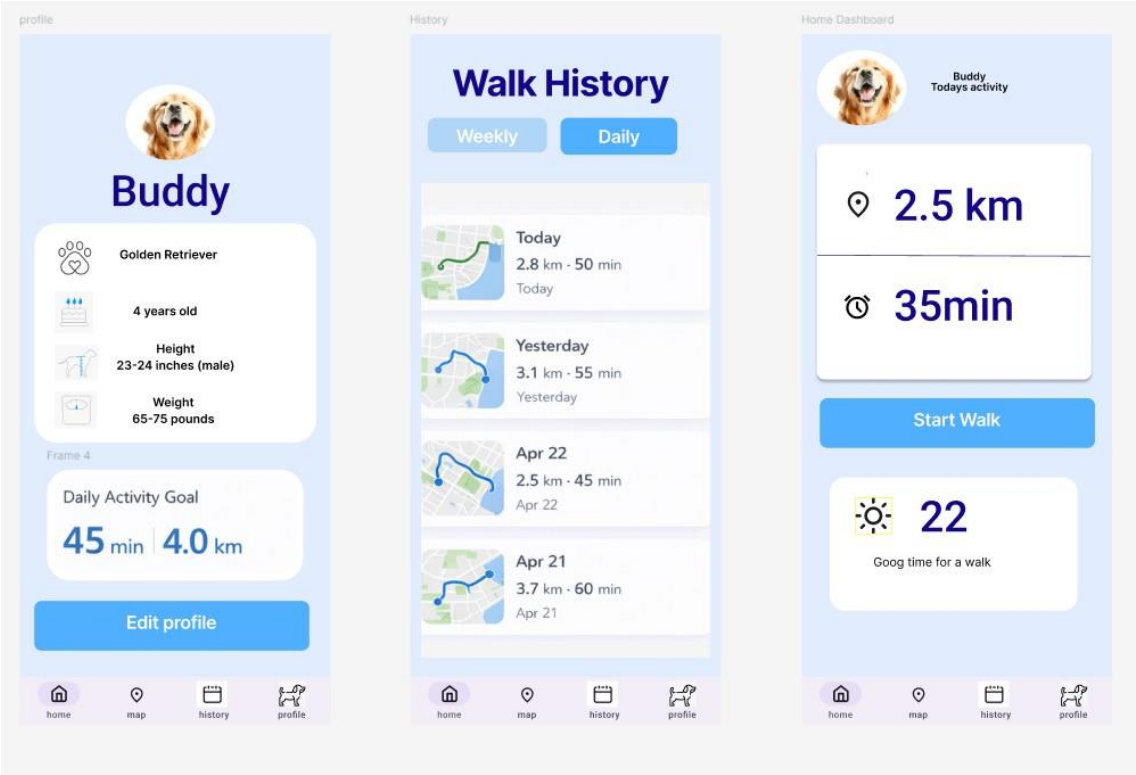
```
├─ ui/
│   ├─ main/
│   │   ├─ MainScreen.kt
│   │   ├─ MainViewModel.kt
│   │   └─ MainUiState.kt
│   │
│   ├─ tracking/
│   │   ├─ TrackingScreen.kt
│   │   ├─ TrackingViewModel.kt
│   │   └─ TrackingUiState.kt
│   │
│   ├─ history/
│   │   ├─ HistoryScreen.kt
│   │   ├─ HistoryViewModel.kt
│   │   └─ HistoryUiState.kt
│   │
│   ├─ components/
│   │   ├─ Buttons.kt
│   │   ├─ Cards.kt
│   │   ├─ Charts.kt
│   │   ├─ InputFields.kt
│   │   └─ ListItems.kt
│   │
├─ navigation/
│   ├─ ScreenNav.kt
│   └─ Screens.kt
```

```
├─ data/
│   ├─ local/
│   │   ├─ DogEntity.kt
│   │   ├─ WalkEntity.kt
│   │   ├─ GPSEntity.kt
│   │   ├─ DogDao.kt
│   │   ├─ WalkDao.kt
│   │   └─ AppDatabase.kt
│   │
│   ├─ remote/
│   │   ├─ WeatherApi.kt
│   │   ├─ RetrofitInstance.kt
│   │   └─ WeatherResponse.kt
│   │
│   └─ repository/
│       ├─ DogRepository.kt
│       ├─ WalkRepository.kt
│       ├─ GPSRepository.kt
│       └─ WeatherRepository.kt
│
├─ location/
│   └─ GPSService.kt
│
├─ utils/
│   ├─ Extensions.kt
│   └─ Constants.kt
```

8 Design

The application follows Material Design principles to ensure clean interface. Decisions focuses on simplicity and responsiveness. Each screen is designed for easy navigation and clear information. UI is designed using Figma.

8.1 Application Layout



8.2 Logo

9 Scrum Process

The application development project follows Scrum methodology adapted for small two person team. Sprints last one week and includes the following: sprint planning, development phase, review and retrospective. Scrum master is responsible for presenting in the reviews and to add the documents to GitHub. Due to small team size roles are shared. One team member acts as Product owner. The Scrum Master role is shared according the following schedule:

Product Owner, Scrum Master 1, Emmi Korhonen (18.3–15.4.2026)

Scrum Master 2, Farnaz Jamilpanah (16.4.–6.5.2026)

Daily scrum meeting are held on every Monday, Wednesday and Friday. Continuous communication is maintained using Discord. Tasks are managed using a product backlog. The project also utilizes a burndown chart to track progress and monitor sprint performance.

10 Team Roles

We divide the work into Backend Developer and Frontend Developer, which is a realistic and efficient way to build an Android app. Team members work on both areas to share responsibilities.

Backend Developer Tasks:

- GPS tracking and location updates
- Route recording logic
- Room database implementation
- Weather API integration (optional)
- Activity analysis (optional AI)

- ViewModel logic
- Unit tests

Frontend Developer Tasks:

- UI design
- User experience
- Jetpack Compose UI design
- Map screen UI
- History and statistics screens
- Navigation between screens
- Icons, colors, accessibility
- UI tests

11 Testing Plan

The development includes testing to ensure the quality, functionality and usability of the application. Features are tested continuously during development. Test used are Unit testing, UI testing, manual testing and final phase is User Acceptance Testing (Optional). Based on the testing results, the application will be further developed and improved.

12 Risks and Mitigation

Technical, schedule and user adaption risks.

Risk 1: GPS: may give inaccurate or delayed location updates; this can cause messy routes or incorrect distance calculations. especially:

- Indoors
- Near tall buildings
- In bad weather
- When battery optimization restricts location updates

Mitigation:

- minimum distance filter
- ignoring low-accuracy points
- Test outdoors in different environments
- Allow user to restart tracking if GPS is unstable

Risk 2: Google Maps Integration Problems: map may not load API key (misconfiguration)

Mitigation:

- Set up API key early in the project
- Test on both emulator and real device
- Keep API key in local.properties (not in GitHub)

Risk 3: Time Management Problems: some features may take longer than expected.

Mitigation:

- Follow the 6-week timeline
- Prioritize core feature first: GPS- Maps- Room
- Add advanced features only if time allows; like AI or notifications
- Keep UI simple at first then publish later

13 Timeline

Sprint	Dates	Focus	Tasks
Sprint 1	25.3-1.4	Map, route	MVVM, screens, Maps API, GPS
Sprint 2	1.-8.4	Room database, history	Room DB, save walks data, show history
Sprint 3	8.-15.4	Statistics	Calculate stats, display charts, test
Sprint 4	15.-22.4	UI improvements	Material Design, UI improvements
Sprint 5	22.-29.4	Testing, bug fixing	Optional features, Unit/UI tests, fix bugs
Sprint 6	29.4-6.5	Documentation, UAT (Optional)	Documentation, User testing

Links

GitHub

https://github.com/meemmi/mobile-app-project

Figma

https://www.figma.com/design/ayZ0S3BYLx7QVEisAuze25/Dog-tracker-App?node-id=0-1&p=f&t=gW0HXCYr0HvJftSZ-0

Jira

metropolia-mobile-app-project